

Firo FCMPs

Luke "Kayaba" Parker

January 15, 2025

Abstract

FCMPs, short for Full-Chain Membership Proofs, are an efficient ZK proof of set-membership premised on Curve Trees. Originally designed for the Monero project's Seraphis upgrade, and later extended into the FCMP++ upgrade, the FCMPs from the FCMP++ upgrade are repurposed here for Firo's deployment of their Lelantus Spark protocol.

1 Background

Lelantus Spark defines an output, as relevant here, as S, O , requiring the re-randomization of S and O over H . This document, derived from the FCMP++ technical document, details a membership proof for that purpose and how its integration into the Firo project would be done. For more explanation of the concepts here, please refer to the original paper, located at <https://github.com/kayabaNerve/fcmp-plus-plus-paper>.

Generalized Bulletproofs, <https://github.com/cyphersstack/generalized-bulletproofs>, implemented at <https://github.com/kayabaNerve/fcmp-plus-plus/tree/develop/crypto/generalized-bulletproofs>, audited in <https://github.com/cyphersstack/generalized-bulletproofs-code> is the R1CS proof planned for use.

The gadgets are inherited from Monero's specifications and implementation. This minimizes independent development, auditing necessary. The notation and specification are also imported from the FCMP++ technical document, providing the following:

- Equality
- Inverse
- Inequality
- Member of List
- On Curve
- Incomplete Addition
- Tuple Member of List

- Discrete Log

Instead of Helios and Selene, a towering curve cycle over Ed25519, Firo's instantiation of Lelantus Spark is over secp256k1. secp256k1, secq256k1 is used as a curve cycle instead.

1.1 Circuit

We define two distinct layer gadgets, one for the first layer (which ensures the integrity of the tuple output), and one for all additional layers.

1.1.1 First Layer

FirstLayer(S', O') $\rightarrow$

$s_x, s_y, r_{s0..256}, r_{s_x}, r_{s_y} \leftarrow \text{Prover (Vector Commitment)}$
 $\text{OnCurve}_{a,b}(s_x, s_y)$
 $\text{DiscreteLog}_H(r_s, r_{s_x}, r_{s_y})$
 $\text{IncompleteAddition}(s_x, s_y, s_{r_x}, s_{r_y}, S'.x, S'.y)$

$c_x, c_y, r_{c0..255}, r_{c_x}, r_{c_y} \leftarrow \text{Prover (Vector Commitment)}$
 $\text{OnCurve}_{a,b}(c_x, c_y)$
 $\text{DiscreteLog}_H(r_c, r_{c_x}, r_{c_y})$
 $\text{IncompleteAddition}(c_x, c_y, r_{c_x}, r_{c_y}, C'.x, C'.y)$

$L \leftarrow \text{Prover (Vector Commitment)}$
 $\text{TupleMemberOfList}(L, (s_x, c_x))$

1.1.2 Additional Layer

AdditionalLayer($\tilde{H}$) $\rightarrow$

$h_x, h_y, r_{0..255}, r_x, r_y \leftarrow \text{Prover (Vector Commitment)}$
 $\text{DiscreteLog}_H(r, r_x, r_y)$
 $\text{OnCurve}_{a,b}(h_x, h_y)$
 $\text{IncompleteAddition}(h_x, h_y, r_x, r_y, \tilde{H}.x, \tilde{H}.y)$

$L \leftarrow \text{Prover (Vector Commitment)}$
 $\text{MemberOfList}(L, h_x)$

Please note $\tilde{H}$ is the prior layer's blinded hash and H is the blinding generator (not the unblinded hash, which is h_x, h_y).

2 Integration

With the circuit defined, it's time to discuss how integration will be handled.

2.1 Tree

The tree is identical to the tree from FCMP++s, with the following caveats.

- The leaf layer flattens S, C , not O, I, C .
- Per 6.2, Monero adds a term with a constant coefficient of 1 to all of the branch hashes within its tree to remove permissibility without allowing a hash collision (idea by tevador). Monero accepts the ability to prove for negative inputs, as the nullifiers for such outputs share an x-coordinate with their originals. By re-defining the nullifier to just the x-coordinate, double-spending isn't possible. While amounts can be negated, this isn't beneficial to an adversary and is solely another way to burn the supply.

Firo cannot accept this as outputs would not have nullifiers linkable to the nullifiers for their negatives. Allowing negative outputs permits double-spending accordingly.

The Firo tree also adds such a term with a constant coefficient to the S, C components prior to hashing and S', C' components prior to passing them to the FCMP. Having $S + T$ and $-S + T + r_s H$ share an x-coordinate when re-randomized over H requires knowing the relationship of S, H which is only possible (given assumptions) if S is solely defined over H . Since the serial number is committed to within S by the generator F , this is only possible if the serial number is 0. If the serial number is 0, the nullifier will be undefined and the Chaum Pedersen proof will require the prover know the relation of U, G to create a valid proof (where U, G are independently sampled points). Accordingly, this prevents spending negated outputs.

Using a term with a constant coefficient on the leaf layer can be credited to Curve Forests, <https://eprint.iacr.org/2024/1647>.

- Monero only grows with outputs with 10 confirmations. Firo will have to define its own n-confirmations to grow/trim the tree upon. n may be 1 if they do not wish to introduce any amount of required confirmations other than inclusion on the blockchain.
- secp256k1 and secq256k1 have to be checked that they don't allow points with zero x-coordinates. If they do, Z needs to be re-defined.

2.2 Modifications to Transaction Verification

Transactions now include:

- **ReferenceBlock:** The block for which the FCMP is premised on.
- **Layers:** The amount of layers in the tree the contained proof is for (necessary to deserialize the proof itself without context)

- FCMP: The FCMP itself, represented as an opaque byte buffer whose length is fixed to `Layers`

`ReferenceBlock` is confirmed to be on the best chain and `n`-blocks deep. The tree root after applying that block is fetched, along with the amount of layers it has. `Layers` is checked to be equivalent. The FCMP verification code is called with the tree root, S', C' and FCMP.

2.3 Modifications to the RPC

The RPC is extended with a route to return the path for an output (by index) within a specified tree. Wallets SHOULD locally build the tree as they scan. Wallets which don't do this SHOULD sample a set of decoys and request the paths for all outputs within the decoy set to hide which path is actually relevant to them.

2.4 Modifications to Wallets

Wallets locally build the tree and/or are updated to the new RPC routes. Wallets no longer call the one-out-of-many prove function yet the FCMP prove function.